

Logic, Shift and Rotate Instructions

Outline

- Logic instructions
- Shift instructions
- Rotate instructions

Logic Instructions

- We can change individual bits in the computer by using logic operations
- The binary values of 0 and 1 are treated as false and true respectively
- When a logic operation is applied to 8 or 16 bit operands, the result is obtained by applying the logic operation at each bit position
- Logic instructions are: AND, OR, XOR and NOT

Mask

- One use of AND, OR and XOR is to selectively modify the bits in the destination
- To do this we construct a source bit pattern known as mask
- The mask bits are chosen so that the corresponding destination bits are modified in the desired way

$B \text{ AND } 1 = B$	$B \text{ OR } 0 = B$	$B \text{ XOR } 0 = B$
$B \text{ AND } 0 = 0$	$B \text{ OR } 1 = 1$	$B \text{ XOR } 1 = \neg B$

Example of Logical Operations

Perform the following logical operations:

1. 1010 1010 **AND** 1111 0000
2. 1010 1010 **OR** 1111 0000
3. 1010 1010 **XOR** 1111 0000
4. **NOT** 1010 1010

AND	OR	XOR	NOT
1010 1010 1111 0000 ----- 1010 0000	1010 1010 1111 0000 ----- 1111 1010	1010 1010 1111 0000 ----- 0101 1010	1010 1010 ----- 0101 0101

Mask

- The AND instruction can be used to clear specific destination bits while preserving the others. A 0 mask bit clears the corresponding destination bit and a 1 mask bit preserves the corresponding destination bit
- The OR instruction can be used to set specific destination bits while preserving the others. A 1 mask bit sets the corresponding destination bit and a 0 mask bit preserves the corresponding destination bit
- The XOR instruction can be used to complement specific destination bits while preserving the others. A 1 mask bit complements the corresponding destination bit and a 0 mask bit preserves the corresponding destination bit

The AND Instruction

- Syntax
 - AND destination, source
- The result of the operation is stored in the destination
- Destination must be a register or memory location
- Source may be a constant, register or memory location
- Memory to memory operations are not allowed
- Effect on flags
 - SF, ZF and PF reflect the result
 - CF, OF = 0
- Converting an ASCII digit to a number and conversion of lowercase letter to upper case letter

Example 1

Clear the sign bit of AL while leaving the other bits unchanged.

Solution: Only need to clear the sign bit or MSB bit and keep the rest as it is. We need to use mask 0111 1111B (07h).

AND AL, 07F

Let's verify the solution for AL = 1010 1100B

AL = 1010 1100

Mask = 0111 1111

0010 1100 (Only the sign bit is changed)

The OR Instruction

- Syntax
 - OR destination, source
- The result of the operation is stored in the destination
- Destination must be a register or memory location
- Source may be a constant, register or memory location
- Memory to memory operations are not allowed
- Effect on flags
 - SF, ZF and PF reflect the result
 - CF, OF = 0
- Testing a register for zero and to check the sign of the value

Example 2

Set the most significant and least significant bits of AL while preserving the other bits.

Solution: Only need to set the MSB and LSB bit and keep the rest as it is. We need to use mask 1000 0001B (81h).

OR AL, 81h

Let's verify the solution for AL = 1010 1100B

AL = 1010 1100

Mask = 1000 0001

1010 1101 (Only the MSB and LSB bits are changed)

The XOR Instruction

- Syntax
 - XOR destination, source
- The result of the operation is stored in the destination
- Destination must be a register or memory location
- Source may be a constant, register or memory location
- Memory to memory operations are not allowed
- Effect on flags
 - SF, ZF and PF reflect the result
 - CF, OF = 0
- Clearing the value of a register

Example 3

Change the sign bit of DX

Solution: Here, we do not know if the sign bit is 0 or 1. To toggle the sign bit, we will XOR with 1. To preserve all other bits, they are XORed with 0. We need to use mask 1000 0000B (80h)

Let's verify the solution for AL = 1010 1100B

AL = 1010 1100

Mask = 1000 0000

0010 1100 (Only the sign bit is changed)

The NOT Instruction

- Syntax
 - NOT destination
- Perform the one's complement operation on the destination
- The result of the operation is stored in the destination
- Destination must be a register or memory location
- There is no effect on the status flags
- Complement the bits of a register or memory location

The TEST Instruction

- Syntax
 - TEST destination, source
- The TEST instruction performs an AND operation of the destination with the source but does not change the destination contents.
- The purpose of the TEST instruction is to set the status flags.
- Effect on flags
 - SF, ZF, PF reflect the result
 - CF, OF= 0

The TEST Instruction

- The TEST instruction can be used to examine individual bits in an operand. The mask should contain 1's in the bit positions to be tested and 0's elsewhere.
- The result of TEST instruction will have 1's in the tested bit positions if and only if the destination has 1's in these positions; it will have 0's elsewhere.
- If destination has 0's in all the tested position, the result will be 0 and so ZF = 1.

Shift Instructions

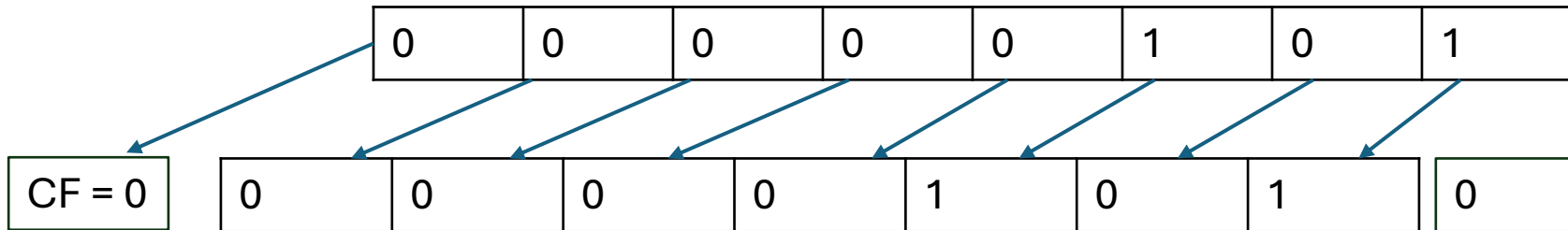
- The shift instructions shift the bits in the destination operand by one or more positions either to the left or right.
- For a shift instruction, the bits shifted out are lost.
- Syntax
 - opcode destination, 1 ;for a single shift
 - opcode destination, CL ;for a shift of N positions where CL contains N.
- In both cases, destination is an 8 or 16-bit register or memory location.
- For intel's more advanced processors, a shift instruction also allows the use of an 8-bit constant.

The SHL Instruction

- The SHL(shift left) instruction shifts the bits in the destination to the left.
- Syntax
 - SHL destination, 1 ;for a single shift
- A 0 is shifted into the rightmost bit position and the MSB is shifted into CF.
- Effect on flags
 - SF, PF, ZF reflect the result
 - CF= last bit shifted out
 - OF= 1 if result changes sign on last shift
- The SHL instruction on a binary number doubles the value

Example

Suppose, AL Contains 5,



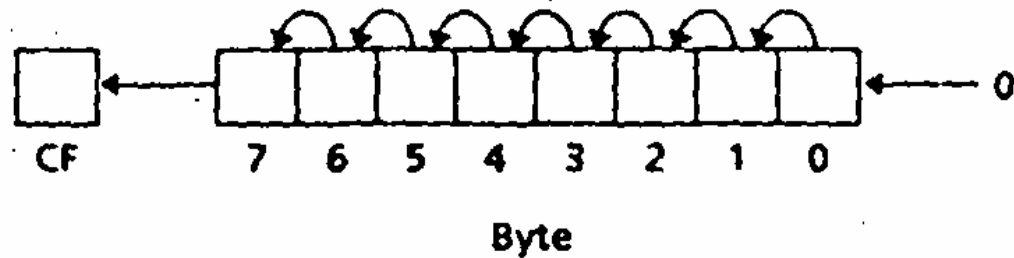
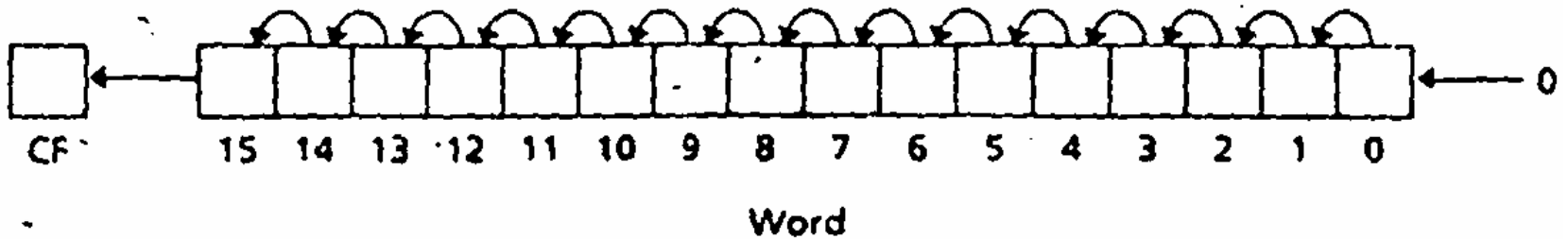
After left shifting by 1, we get 10.

If we left shift it again, we will get 20 ($5 * 2^2$) = 20

The SAL Instruction

- The opcode SAL (shift arithmetic left) is often used in instances where numeric multiplication is intended.
- SAL instructions generate the same machine code as SHL instruction.
- Negative numbers can also be multiply by powers of 2 by left shifts
- For example, if AX is FFFFh (-1), then shifting three times will yield AX= FFF8h (-8).

How SHL and SAL works



Overflow

- When we treat left shifts as multiplication, overflow may occur.
- For a single left shift, CF and OF accurately indicate unsigned and signed overflow, respectively.
- But the overflow flags are not reliable indicators for a multiple left shift.
- This is because a multiple shift is really a series of single shifts, and CF, OF only reflect the result of the last shift.
- For example, if BL contains 80h, CL contains 2 and we execute SHL BL,CL, then CF = OF = 0 even though both signed and unsigned overflow occur.

Example 4

Write some code to multiply the value of AX by 8. Assume that overflow will not occur.

Solution: To multiply by 8, we need to do three left shifts.

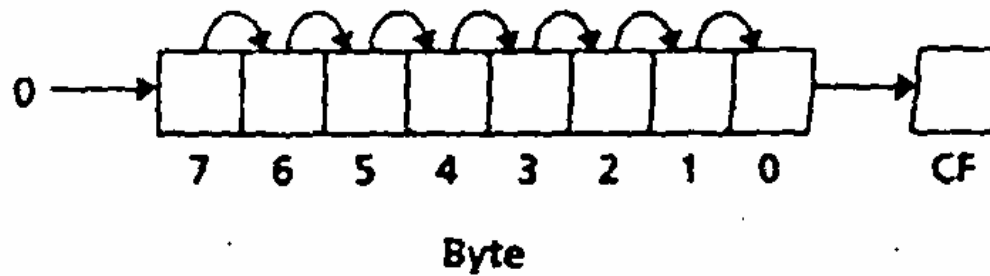
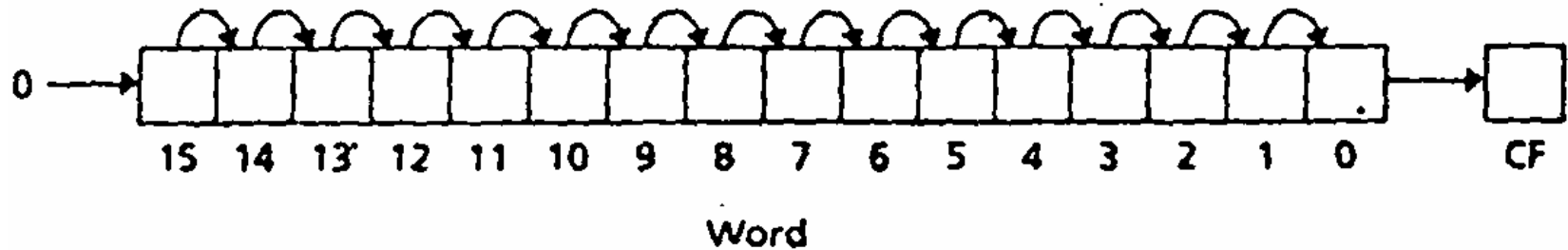
MOV CL,3; number of shifts to do

SAL AX,CL; multiplyby8

The SHR Instruction

- The instruction SHR (shift right) performs right shifts on the destination operand.
- Syntax
 - SHR destination, 1 ;for a single shift
 - SHR destination, CL ;where CL contains N.
- A 0 is shifted into the MSB position, and the rightmost bit is shifted into CF.
- Effect on flags
 - SF, PF, ZF reflect the result
 - CF= last bit shifted out
 - OF= 1 if result changes sign on last shift
- The SHR instruction on a binary number halves the value if it is an even number. For odd numbers, a right shift halves it and rounds down to the nearest integer.

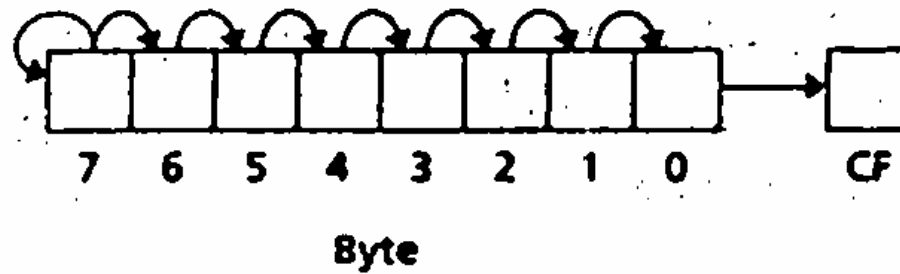
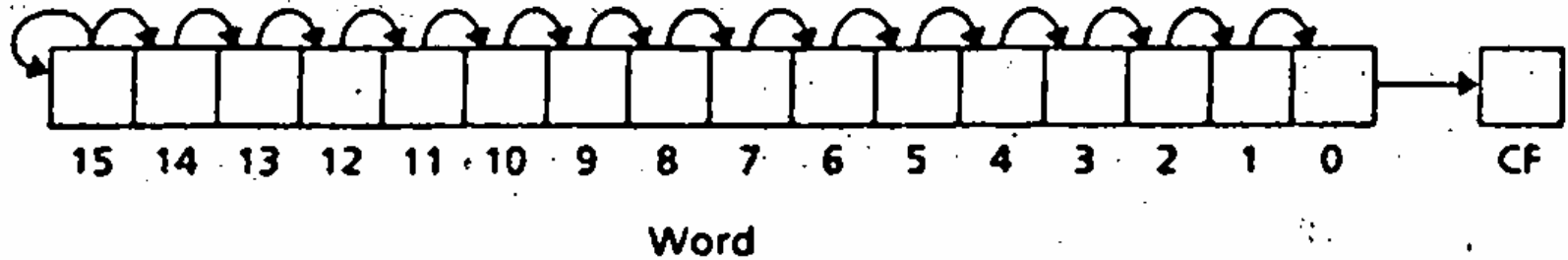
How SHR Works



The SAR Instruction

- The SAR Instruction (shift arithmetic right) operates like SHR
- The MSB retains its original value.
- Syntax
 - SAR destination, 1 ;for a single shift
 - SAR destination, CL ; where CL contains N.
- Effect on flags
 - SF, PF, ZF reflect the result
 - CF= last bit shifted out
 - OF= 1 if result changes sign on last shift

How SAR Works



Division using SHR and SAR

To do division by right shifts, we need to make distinction between signed and unsigned number.

- For unsigned interpretation, SHR should be used.
- For signed interpretation, SAR should be used because it preserves the sign.

If AL contains -15, give the decimal value of AL after SAR AL, 1 is performed.

Execution of SAR AL, 1 divides the number by 2 and rounds down. Dividing -15 by 2 yields -7.5, and after rounding down we get -8.

Self-Study

1. Suppose BH contains 8Ah and CL contains 3. What are the values of BH and of CF after the instruction SHL BH,CL is executed? (Ans: BH = 01010000b = 50h, CF = 0)
2. Suppose BH contains 8Ah and CL contains 2. What are the values of BH and CF after the Instruction SHR BH,CL is executed? (Ans: BH = 00100010b = 22h, CF = 1)

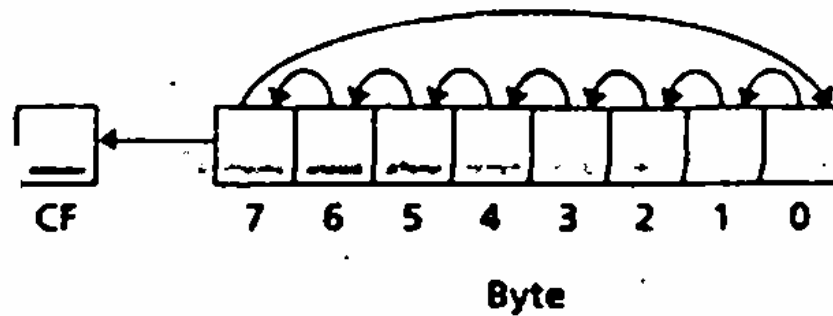
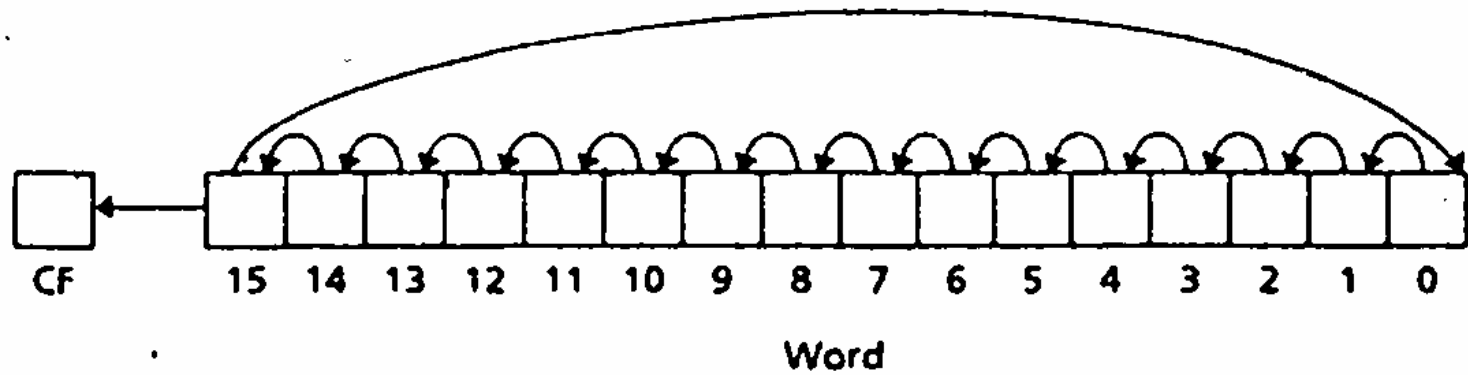
Rotate Instructions

- The rotate instructions rotate the bits in the destination operand by one or more positions either to the left or right.
- For a rotate instruction, bits shifted out from one end of the operand are put back into the other end.
- Syntax
 - opcode destination, 1 ;for a single rotate
 - opcode destination, CL ;for a rotate of N positions where CL contains N.
- In both cases, destination is an 8 or 16-bit register or memory location.
- For intel's more advanced processors, a rotate instruction also allows the use of an 8-bit constant.

The ROL Instruction

- The instruction ROL (rotate left) shifts bits to the left.
- The MSB is shifted into the rightmost bit.
- The CF also gets the bit shifted out of the MSB.
- Destination bits forming a circle, with the least significant bit following the MSB in the circle.
- Syntax
 - ROL destination, 1
 - ROL destination, CL
- In ROL, CF reflects the bit that is rotated out. This can be used to inspect the bits in a byte or word without changing the contents.

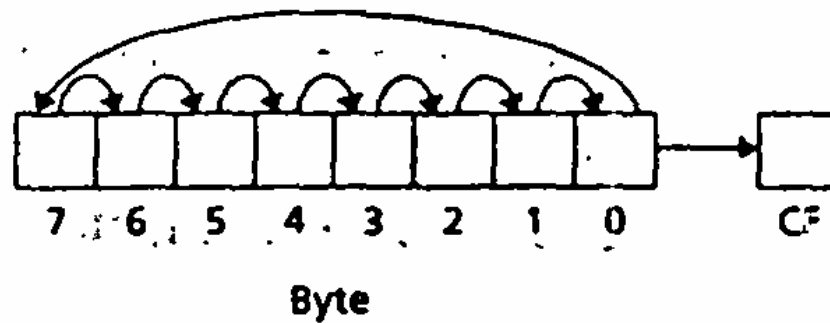
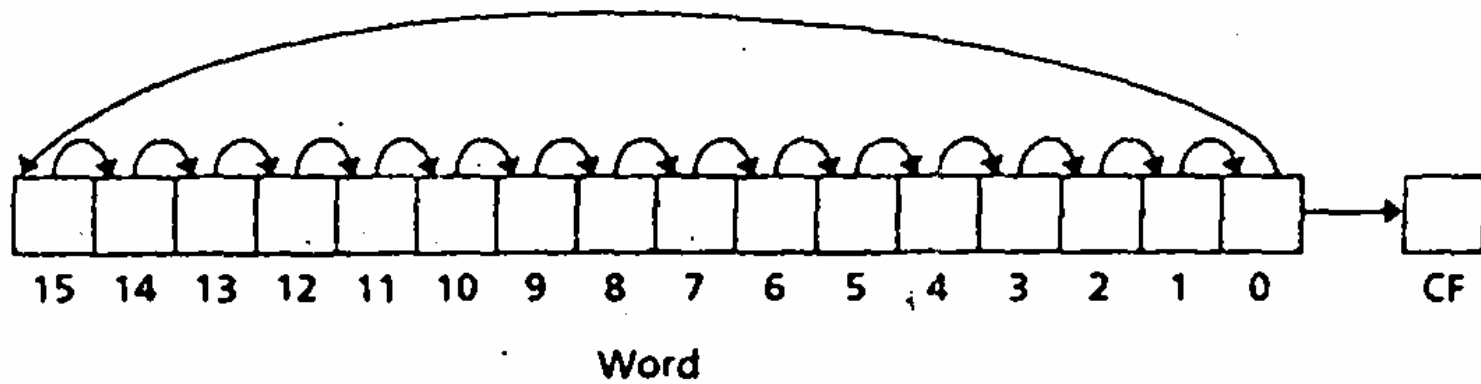
How ROL Works



The ROR Instruction

- The instruction ROR (rotate right) shifts bits to the right.
- The rightmost bit is shifted into the MSB and also into the CF.
- Syntax
 - ROR destination, 1
 - ROR destination, CL
- In ROR, CF reflects the bit that is rotated out. This can be used to inspect the bits in a byte or word without changing the contents.

How ROR Works



Example

- Use ROL to count the number of 1 bits in BX, without changing BX. Put the answer in AX.

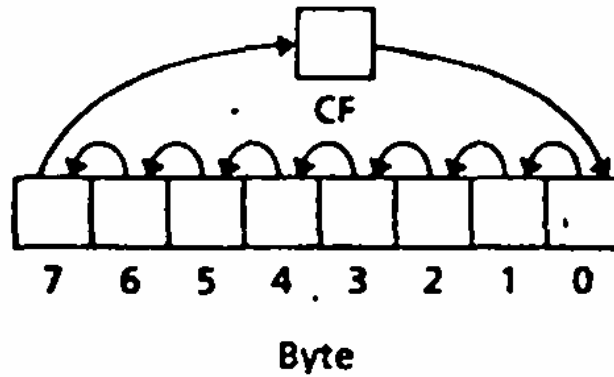
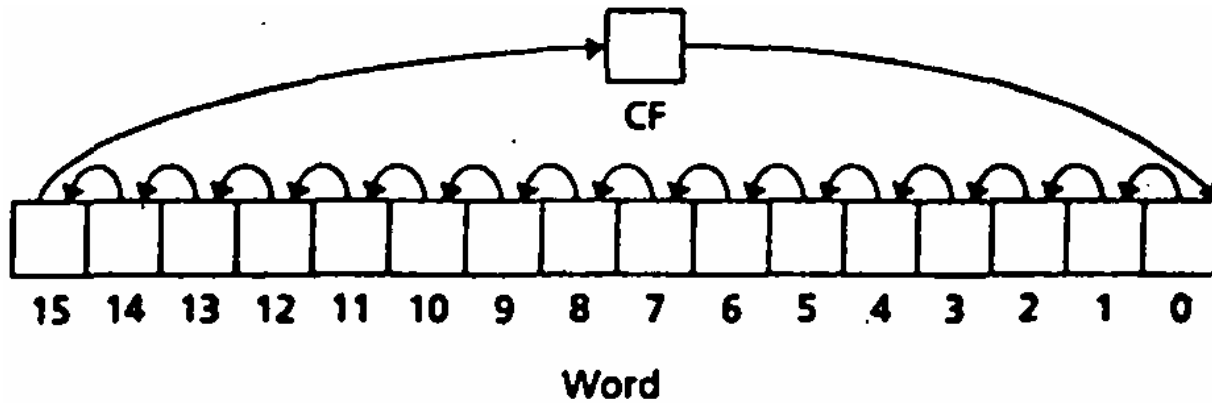
- Solution:

```
XOR  AX,AX      ;AX counts bits
MOV  CX, 16     ;loop counter
TOP:
ROL  BX,1       ;CF=bit rotated out
JNC  NEXT       ;0 bit
INC  AX         ;1 bit, increment total
NEXT:
LOOP TOP        ;loop until done
```

The RCL Instruction

- The instruction RCL (Rotate through Carry Left) shifts the bits of the destination to the left.
- The MSB is shifted into the CF, and the previous value of CF is shifted into the rightmost bit.
- RCL works like ROL, except that CF is part of the circle of bits being rotated.
- Syntax
 - RCL destination, 1
 - RCL destination, CL
- Effect on the flags
 - SF, PF, ZF reflect the result
 - CF = last bit shifted out
 - OF = 1 if result changes sign in the last rotation

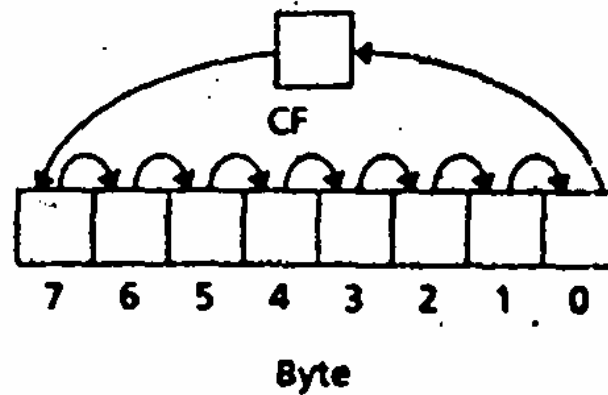
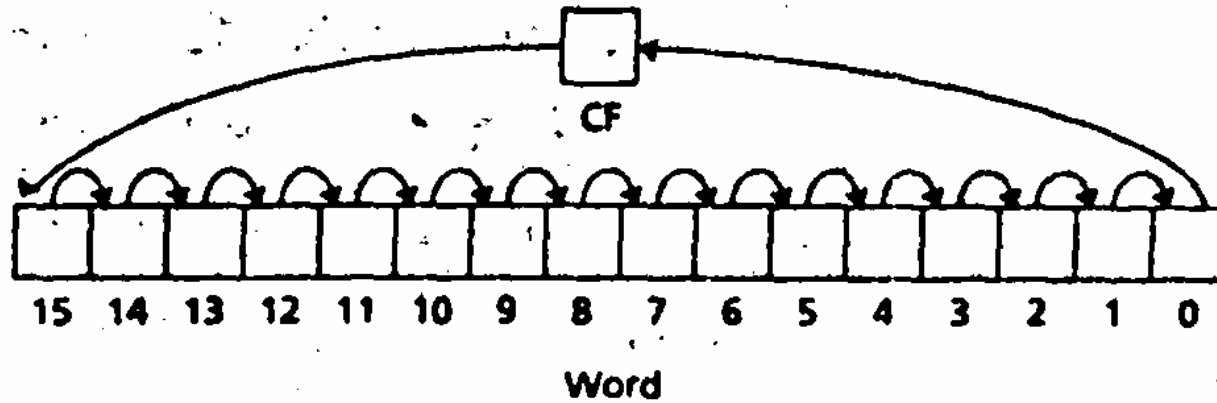
How RCL Works



The RCR Instruction

- The instruction RCR (Rotate through Carry Right) shifts the bits of the destination to the right.
- The LSB is shifted into the CF, and the previous value of CF is shifted into the leftmost bit.
- RCR works like ROR, except that CF is part of the circle of bits being rotated.
- Syntax
 - RCR destination, 1
 - RCR destination, CL
- Effect on the flags
 - SF, PF, ZF reflect the result
 - CF = last bit shifted out
 - OF = 1 if result changes sign in the last rotation

How RCR Works



Example

Suppose BH contains 8Ah, CF= 1, and CL contains 3. What are the values of BH and CF after the instruction RCR BH,CL is executed?

	CF	BH
Initial Values	1	1000 1010
After 1 st rotation	0	1100 0101
After 2 nd rotation	1	0110 0010
After 3 rd rotation	0	1011 0001

Example

Reversing a bit pattern

For example, if AL contains 11011100, we want to make 00111011

```
REVERSE_:  
SHL AL, 1  
RCR BL, 1  
LOOP REVERSE_  
MOV AL, BL
```

Use SHL to shift the bits out the left end of AL
Into CF and then use RCR to move them Into
the left end of another register BL;

Lastly copy the contents of BL from AL